

Lecture 1: Introduction to Neural Networks

Neural Networks Course (MC459)

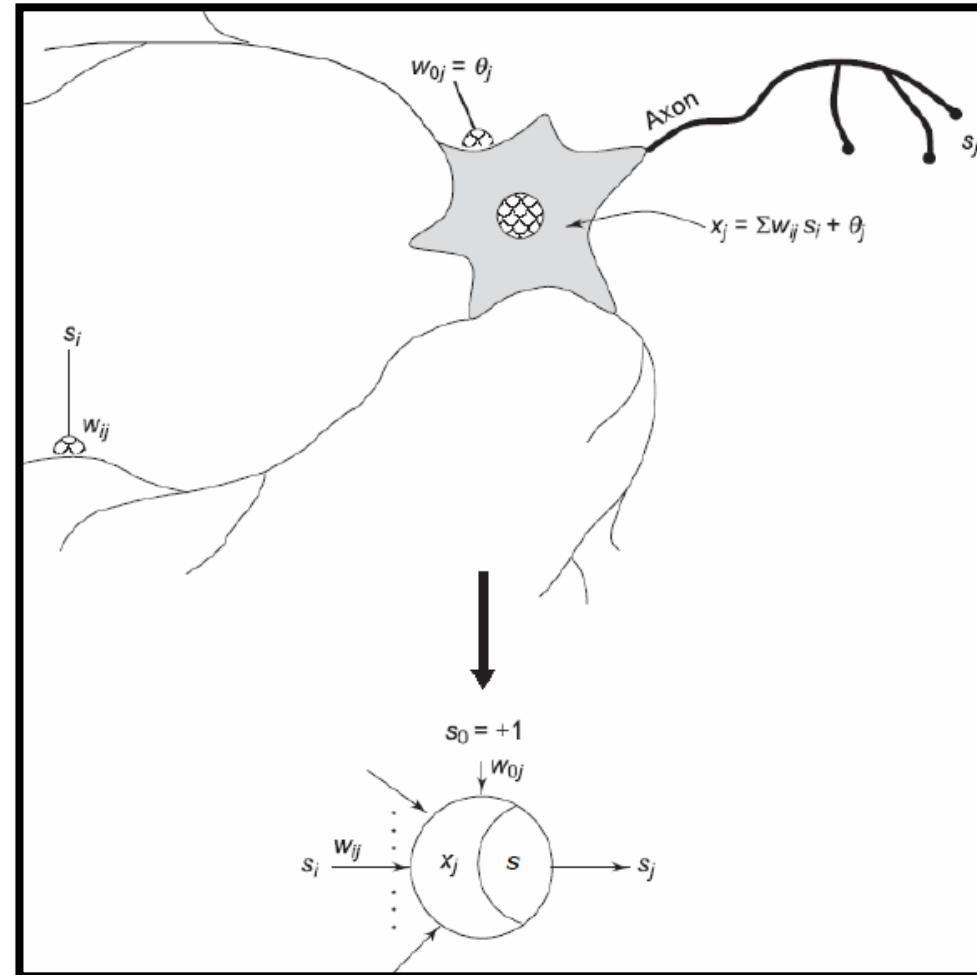
Dr. Amal Abd-Elazim

What is a Neural Network?

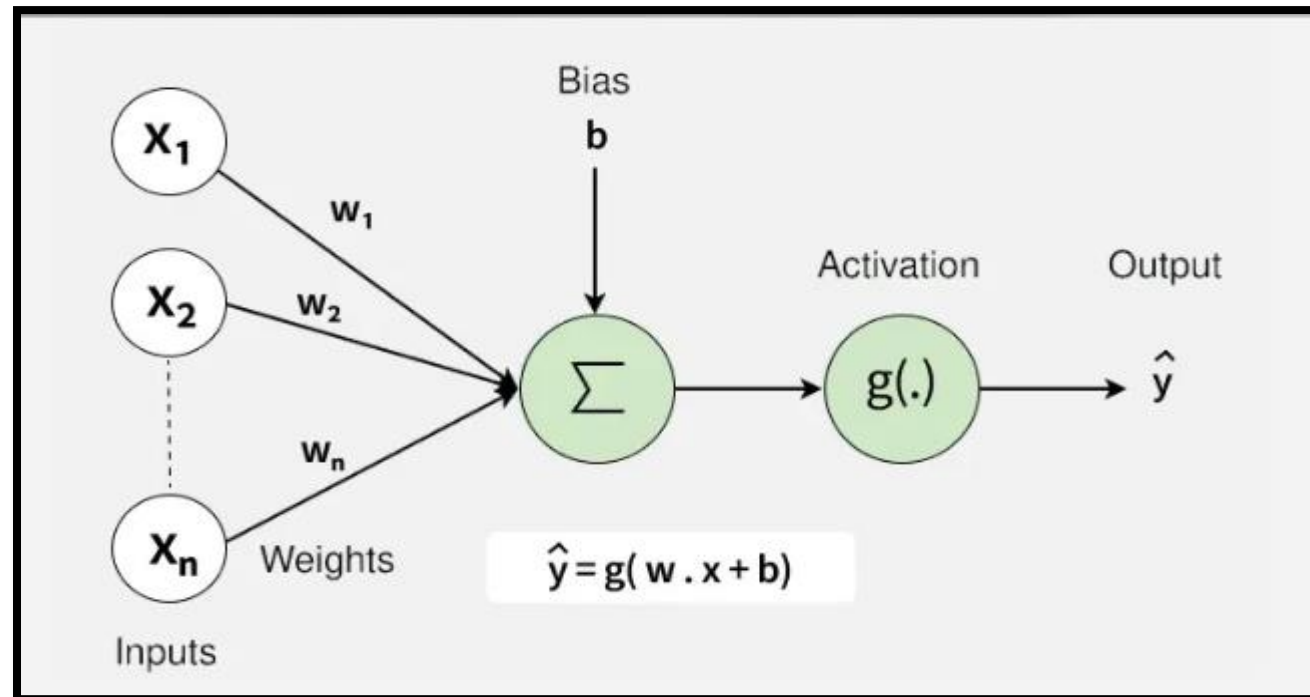
- **Neural networks** are machine learning models that **mimic** the complex functions of the **human brain**. These models consist of interconnected nodes called **neurons** that process data, learn patterns and enable tasks such as pattern recognition and make decisions, or predict outcomes based on data.

Neuron Abstraction

- Neurons send signals by changing them from electrical to chemical and then back to electrical again.
- Each connection between two neurons, called a synapse, has something known as synaptic efficacy. This means how well the signal is passed from the first neuron (presynaptic) to the second one (postsynaptic).

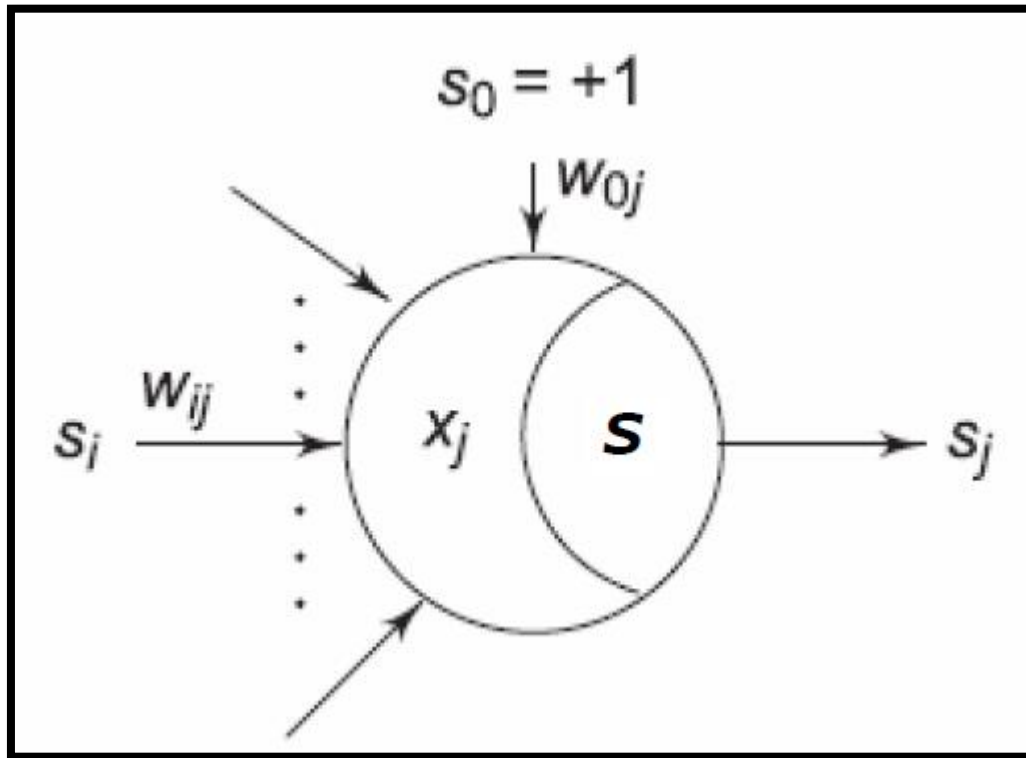


Artificial Neuron



Artificial Neuron

- j^{th} artificial neuron that receives input signals s_i , from possibly n different sources.



$$x_j = \sum_{i=1}^n w_{ij} s_i + \theta_j$$

Artificial Neuron

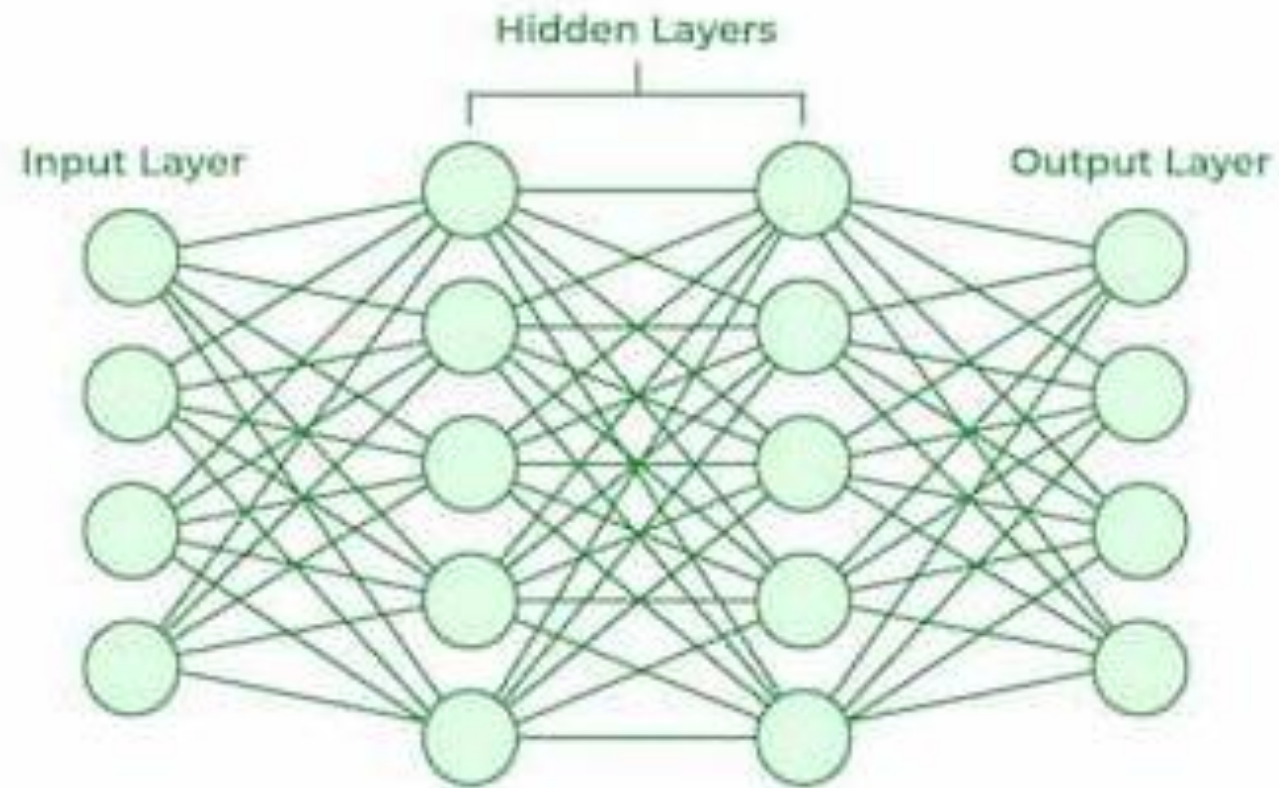
- x_j is a linear weighted aggregation of the impinging signals, modified by an internal threshold, θ_j
- connection weights w_{ij} model the synaptic efficacies of various interneuron synapses.
- w_{ij} denotes the weight from neuron i to neuron j

$$x_j = \sum_{i=1}^n w_{ij} s_i + \theta_j$$

1. Structure:

- A neural network is made up of layers of interconnected units called **neurons** (also called **nodes**). It typically includes:
- **Input Layer:** Receives the raw data (e.g., pixel values of an image).
- **Hidden Layers:** Perform complex transformations and feature extraction.
- **Output Layer:** Produces the final result (e.g., class label, value prediction).

1. Structure:



2. Neuron Function:

- Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through an **activation function** (e.g., ReLU, sigmoid, tanh) to introduce non-linearity.

3. Learning Process:

- Neural networks learn through a process called **training**, which involves:
 - **Forward propagation**: Computing output from inputs.
 - **Loss calculation**: Measuring error between predicted and actual outputs.
 - **Backpropagation**: Adjusting weights to minimize the error using algorithms like **gradient descent**.

4. Types:

- **Feedforward Neural Network (FNN):** Data flows in one direction.
- **Convolutional Neural Network (CNN):** Used in image processing.
- **Recurrent Neural Network (RNN):** Suitable for sequential data like text or time series.
- **Deep Neural Network (DNN):** A network with multiple hidden layers.

Example Use Cases:

- Image and speech recognition
- Natural language processing (NLP)
- Medical diagnosis
- Financial forecasting
- Self-driving cars

Activation (Signal) Function

- After computing the total input, the neuron's **activation** is passed through an **activation function** (also known as a **signal function**) denoted by **S(x)**.
- This function determines the **output signal** of the neuron:

$$s_j = S(x_j)$$

where x_j is the total input to neuron j , and s_j is the output signal.

Activation (Signal) Function

Common types of **activation functions** include:

- **Binary Threshold Function** — outputs 0 or 1 based on a threshold.
- **Linear Threshold Function** — combines linearity and thresholding.
- **Sigmoid Function** — smooth, nonlinear function often used in logistic regression.
- **Gaussian Function** — bell-shaped function useful in radial basis function networks.
- **Probabilistic Function** — outputs a probability value (often used in probabilistic models).

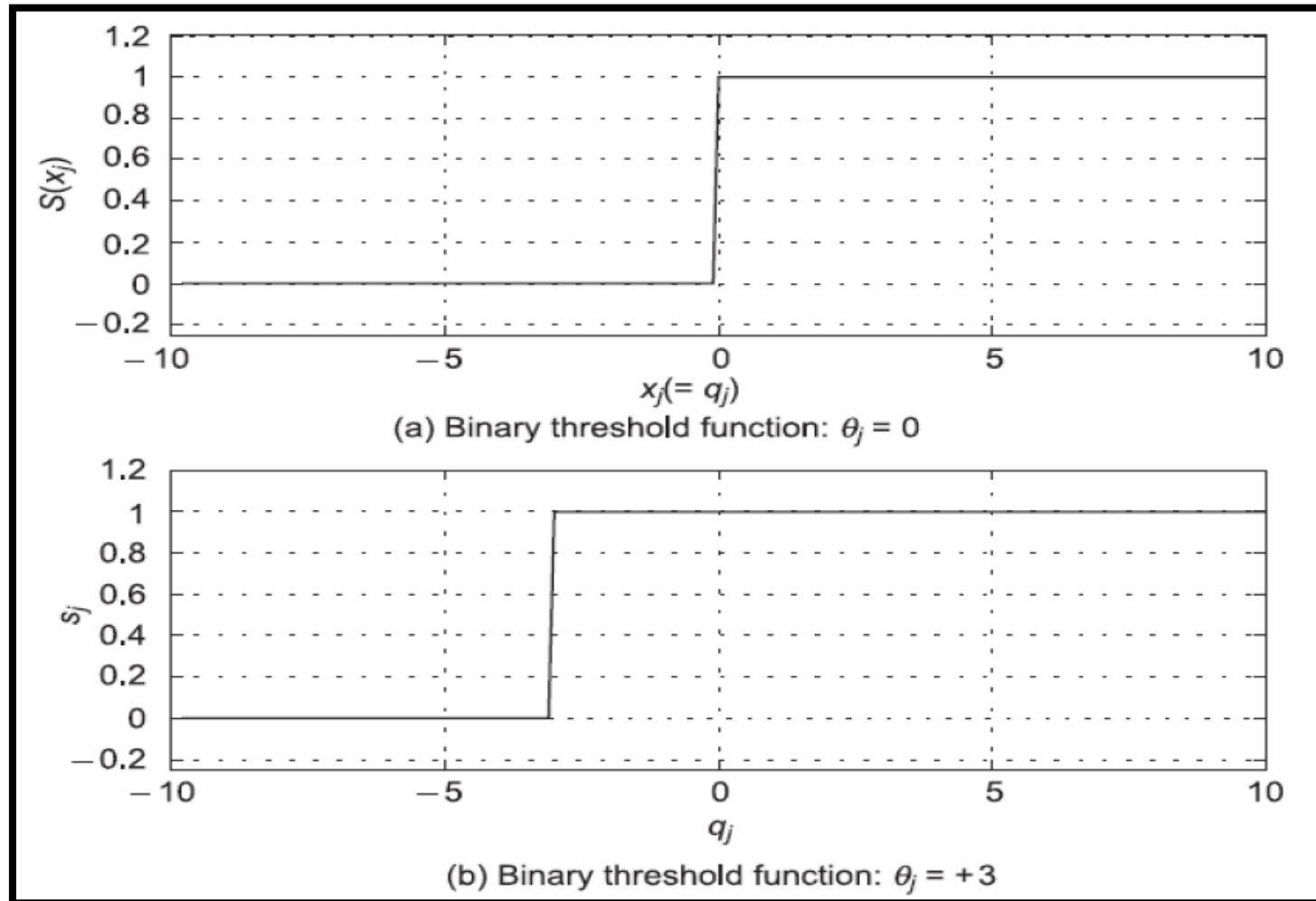
Activation (Signal) Function

- In modern neural networks, popular activation functions also include:
 - **ReLU** (Rectified Linear Unit):
$$S(x) = \max(0, x)$$
 - **Softmax**: used for multi-class classification tasks.
 - **Tanh**: similar to sigmoid but outputs in range $[-1, 1]$.

Neuron Activation Functions: Binary Threshold Function

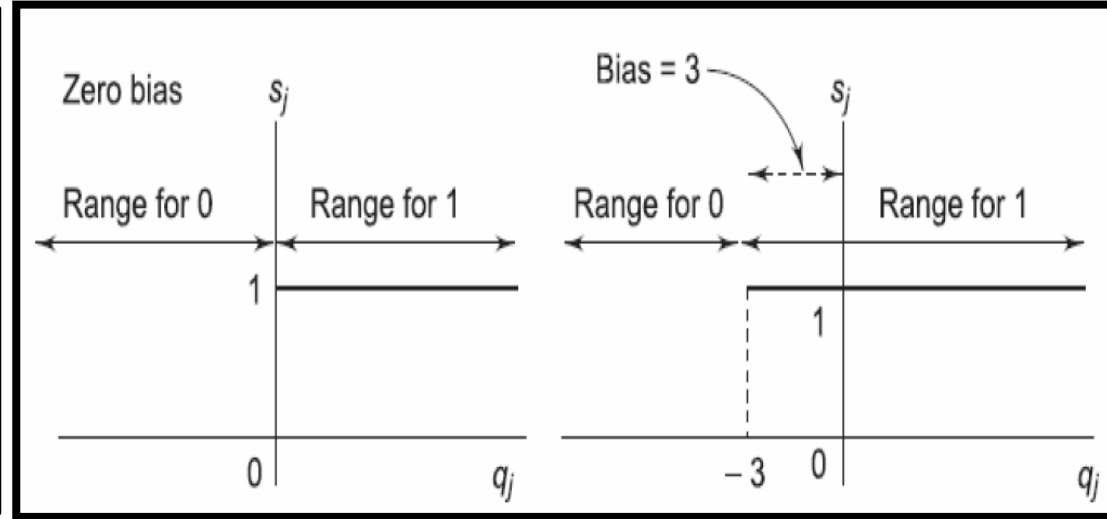
- The **binary threshold function** is one of the simplest activation functions used in neural networks.
- **How it works:**
 - If the total input to the neuron x_j is **greater than or equal to 0**, the output is **1**.
 - If the total input x_j is **less than 0**, the output is **0**.
 - $$S(x_j) = \begin{cases} 1 & \text{if } x_j \geq 0 \\ 0 & \text{if } x_j < 0 \end{cases}$$
- **Key Characteristics:**
 - It produces outputs of **either 0 or 1** only.
 - It creates a "**two-state**" system — like an ON/OFF switch.
 - This is why it's called a **threshold logic unit**.

Neuron Activation Functions: Binary Threshold Function



Interpretation of Threshold

- $$\begin{aligned}x_j &= \sum_{i=0}^n w_{ij} s_i \\&= \sum_{i=1}^n w_{ij} s_i + w_{0j} \\&= q_j + \theta_j\end{aligned}$$



From the neuron's point of view, its **net activation** x_j is calculated as:

$$x_j = q_j + \theta_j$$

- The neuron **fires** (outputs **+1**) if the net input $x_j \geq 0$, which means:

$$q_j \geq -\theta_j$$

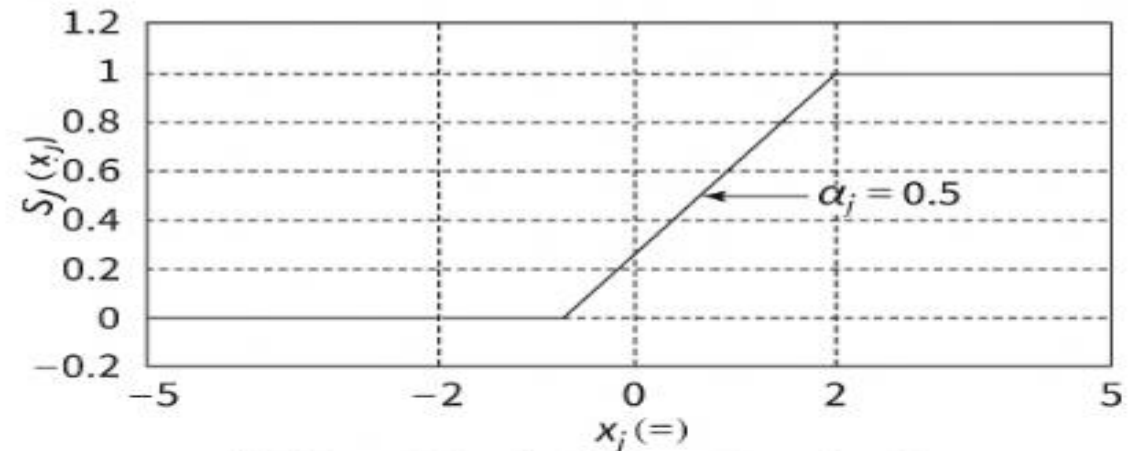
- The neuron outputs **0** if:

$$q_j < -\theta_j$$

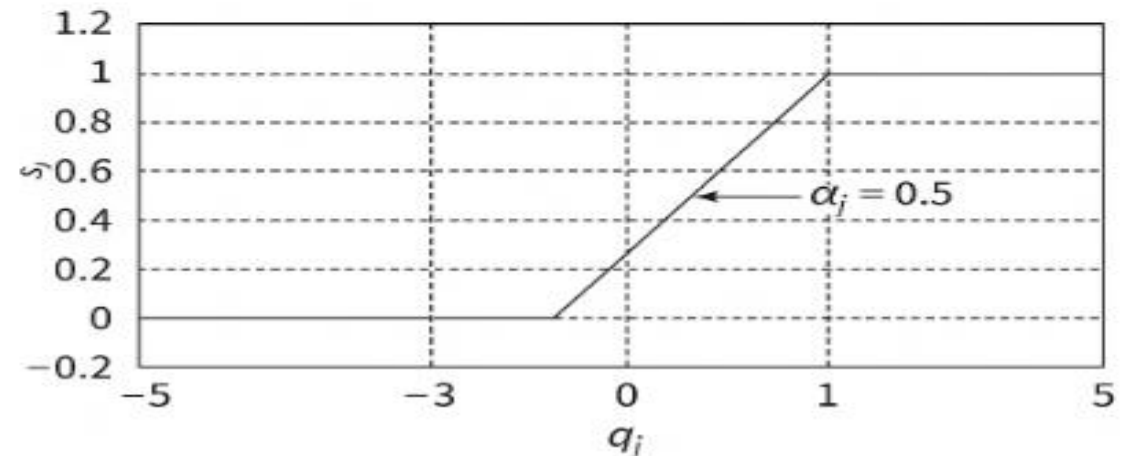
In simple terms, the neuron is **comparing** the input q_j to a **threshold** (specifically, the negative of the bias θ_j)

Linear Threshold Activation Function

- $$S_j(x_j) = \begin{cases} 0 & x_j \leq 0 \\ \alpha_j x_j & 0 < x_j < x_m \\ 1 & x_j \geq x_m \end{cases}$$
- $\alpha_j = 1/x_m$ is the slope parameter of the function.
- Figure plotted for x_m and $\alpha_j = 0.5$.
- $S_j(x_j) = \max(0, \min(\alpha_j x_j, 1))$
- Note that in this course we assume that neurons within a network are homogeneous.



(a) Linear Threshold function: $\theta_j = 0$

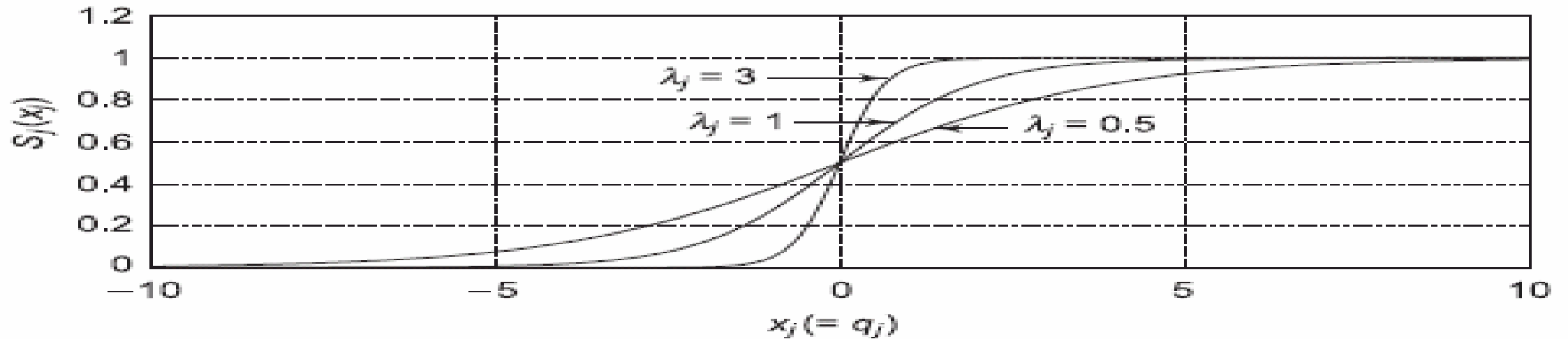


(b) Shifted Linear Threshold function: $\theta_j = -1$

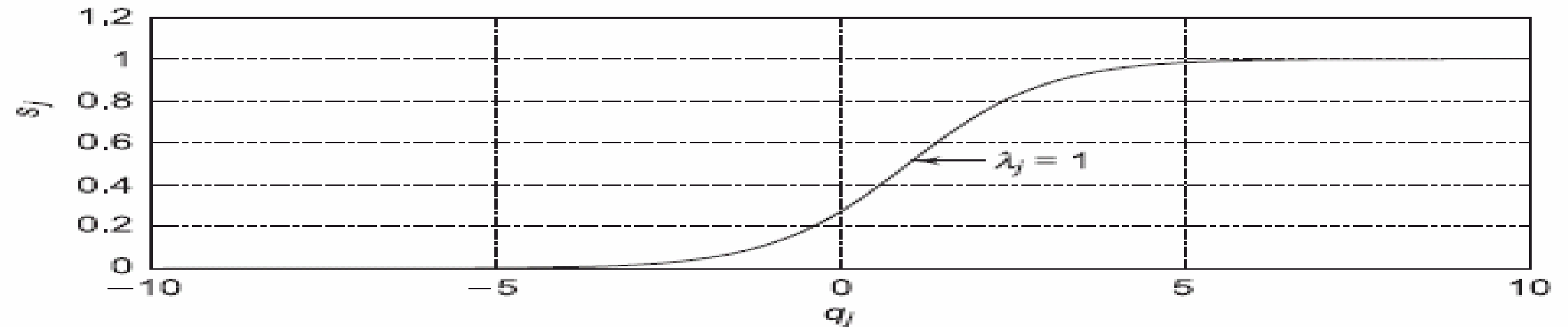
Sigmoidal Activation Function

- $S_j(x_j) = \frac{1}{1+e^{-\lambda_j x_j}}$
- λ_j is a gain scale factor
- In the limit, as $\lambda_j \rightarrow \infty$ the smooth logistic function approaches the non-smooth binary threshold function.
- **The sigmoid function has some useful mathematical features:**
 - It always increases (monotonic)
 - It has no sudden jumps (continuous)
 - Its values stay within a fixed range, usually between 0 and 1 (bounded)

Sigmoidal Activation Function



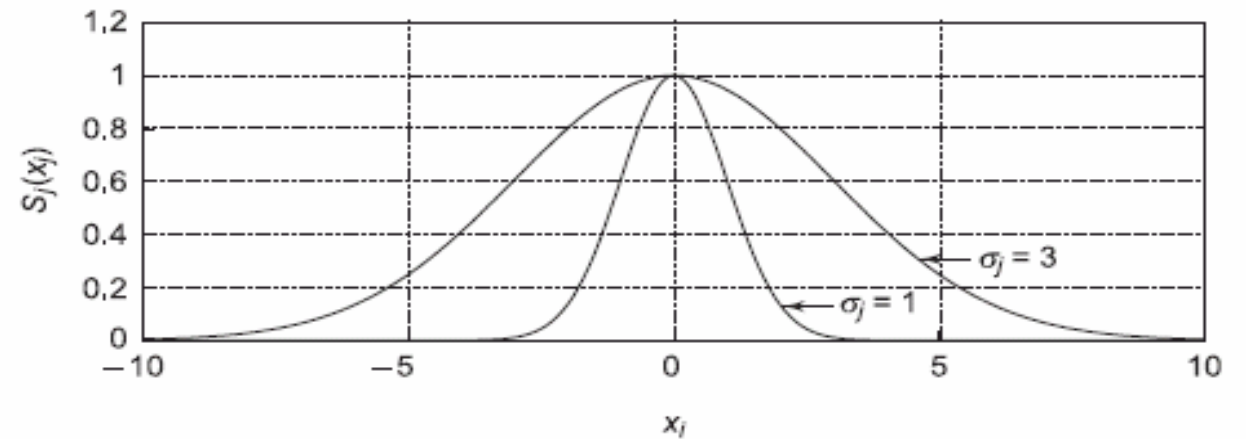
(a) Sigmoidal signal function $\theta_j = 0$



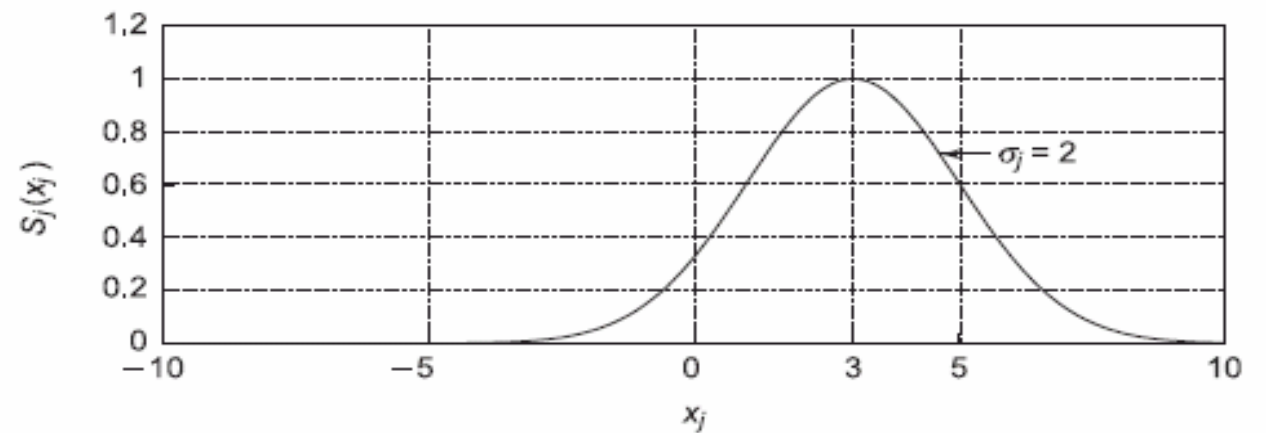
(b) Shifted sigmoidal signal function $\theta_j = -1$

Gaussian Activation Function

- $S_j(x_j) = \exp\left(-\frac{(x_j - c_j)^2}{2\sigma_j^2}\right)$
- σ_j is the Gaussian spread factor and c_j is the center.
- Varying the spread makes the function sharper or more diffuse.
- Changing the center shifts the function to the right or left along the activation axis
- This function is an example of a non-monotonic activation function



(a) Gaussian signal function: center = 0



(b) Gaussian signal function: center = 3

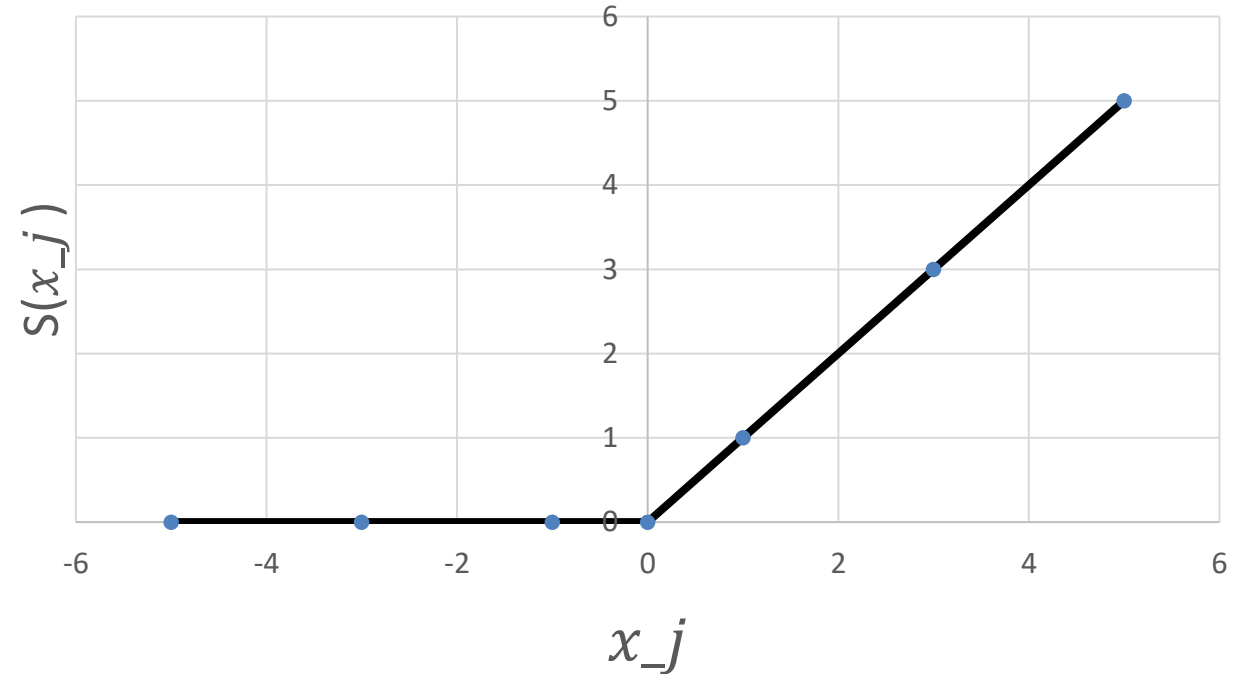
Stochastic Neurons

- The signal is assumed to be two state
 $S_j \in \{0,1\}$ or $\{-1,1\}$
- Neuron switches into these states depending upon a probabilistic function of its activation value, $P(x_j)$.

$$P(x_j) = \frac{1}{1 + e^{-x_j/T}}$$

ReLU – Rectified Linear Unit

- Definition:
 $\text{ReLU}(x_j) = \max(0, x_j)$
- Explanation:
 - Outputs 0 if the input is negative.
 - Outputs the same value if input is positive.



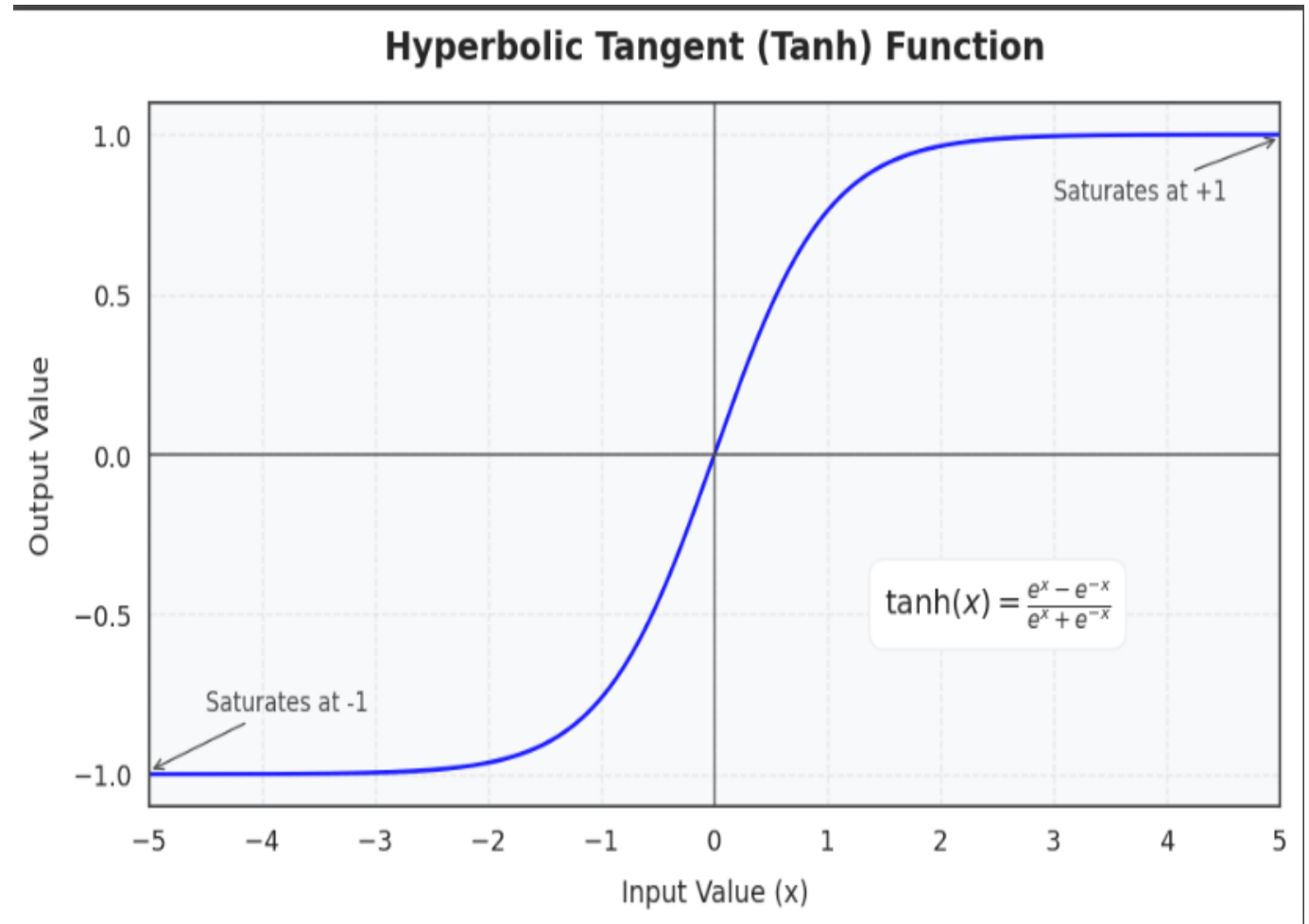
ReLU – Characteristics

- Simple and fast to compute.
- Helps reduce the vanishing gradient problem.
- Not bounded (can output very large values).
- **Drawback:**
 - "Dead neurons" – once output is 0, it might stay 0 forever.

Tanh Activation function

- Tanh (hyperbolic tangent) is a type of activation function that transforms its input into a value between -1 and 1.

- $f(x) = \tanh(x)$
$$= \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh(x)}{\cosh(x)}$$



Comparison Table

Property	ReLU	Sigmoid	Tanh
Output Range	$[0, \infty)$	$(0, 1)$	$(-1, 1)$
Zero-centered	✗	✗	✓
Non-linear	✓	✓	✓
Gradient Issues	Dying ReLU	Vanishing Gradient	Vanishing Gradient
Speed	Fast	Slower	Slower

When to Use Each

- **ReLU:** Most common, best for hidden layers in deep networks.
- **Sigmoid:** Used for binary classification in the output layer.
- **Tanh:** Can be better than sigmoid for hidden layers when zero-centered output is helpful.

Summary of the most common Activation function

- Activation functions are key to learning in neural networks.
- ReLU is fast and simple, but not perfect.
- Sigmoid and Tanh are smooth but suffer from gradient issues.
- Choosing the right function depends on the task.

Eight Components of Neural Networks

- **Neurons.** These can be of three types:
 - *Input:*
 - Receive external data or stimuli from the environment.
 - They serve as the entry point to the network.
 - *Hidden:* compute intermediate functions, They extract patterns and features from the input data.
 - *Output:* Generate the final results or predictions of the network.
- **Activation state vector.** This is a vector of the activation level x_i of individual neurons in the neural network,
$$X = (x_1, \dots, x_n)^T \in R^n.$$
Each x_i is the activation value of the i^{th} neuron.
- **Signal function (Activation function).** A function that generates the output signal of the neuron based on its activation.

Eight Components of Neural Networks

- **Neurons.** These can be of three types:
 - *Input:*
 - Receive external data or stimuli from the environment.
 - They serve as the entry point to the network.
 - *Hidden:* compute intermediate functions, They extract patterns and features from the input data.
 - *Output:* Generate the final results or predictions of the network.
- **Activation state vector.** This is a vector of the activation level x_i of individual neurons in the neural network,
$$X = (x_1, \dots, x_n)^T \in R^n.$$
Each x_i is the activation value of the i^{th} neuron.
- **Signal function (Activation function).** A function that generates the output signal of the neuron based on its activation.

Eight Components of Neural Networks

- **Pattern of Connectivity**
 - This defines how neurons are connected to each other in the network (structure of the neural network). Each connection between neurons can have different effects:
 - Excitatory (+): increases the activation of the next neuron.
 - Inhibitory (-): decreases the activation of the next neuron.
 - Absent (0): no connection between the neurons.
- **Activity Aggregation Rule**
 - This rule explains **how a neuron collects and combines input signals** from other neurons.
 - It is usually calculated as:
 - The **dot product** (inner product) between:
 - The **input vector** (all incoming signals)
 - The **weight vector** (connection strengths)
 - The result determines the **total input** received by the neuron before applying the activation function.

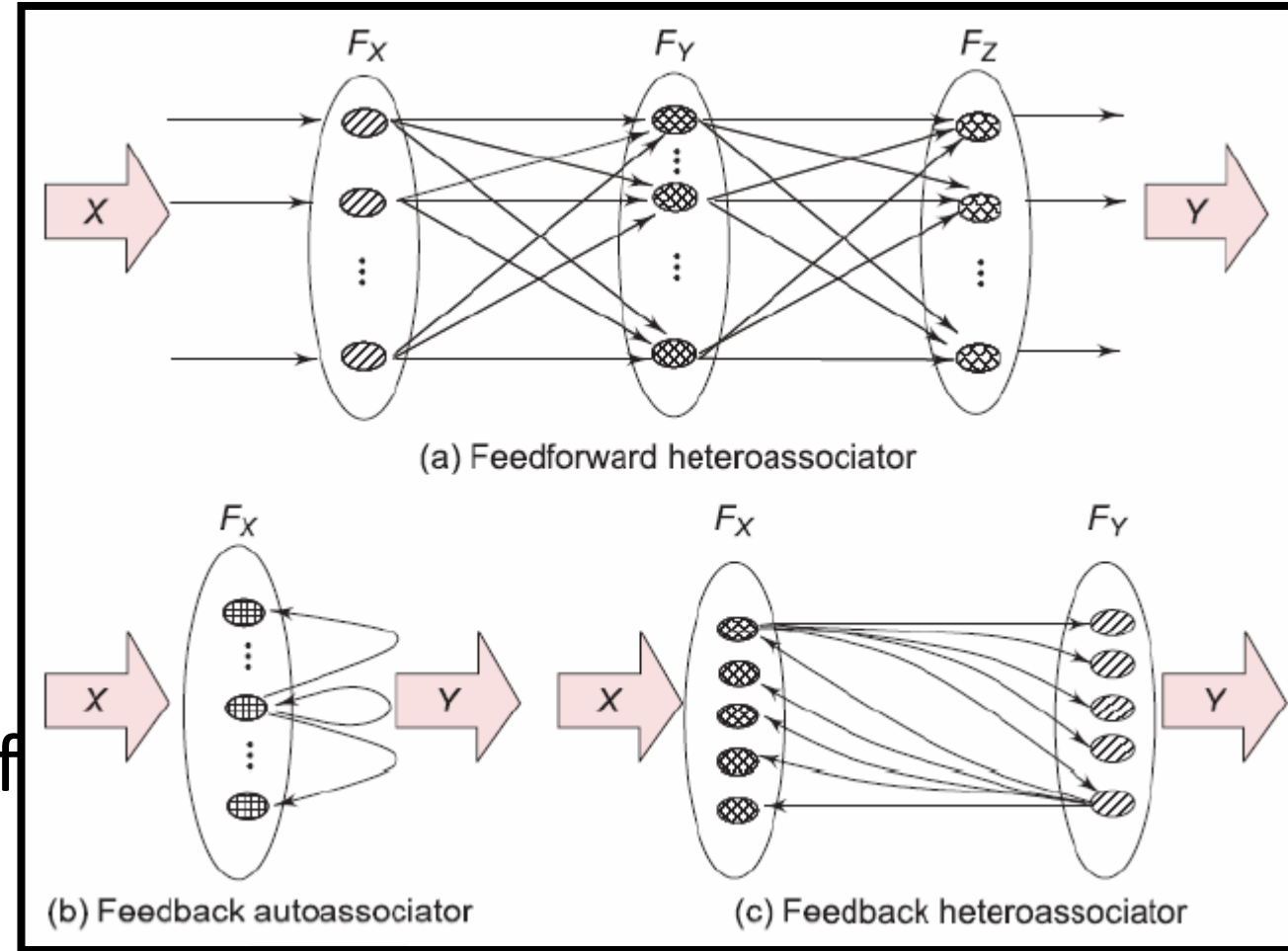
Eight Components of Neural Networks

- **Activation rule (Activation function).** A function that determines the new activation level of a neuron on the basis of its current activation and its external inputs.
- **Learning rule.** Provides a means of modifying connection strengths based both on external stimuli and network performance with an aim to improve the latter.
- **Environment.** The environments within which neural networks can operate could be:
 - deterministic (noiseless) or
 - stochastic (noisy).

Architectures:

Feedforward and Feedback

- Local groups of neurons can be connected in either,
- a feedforward architecture, in which the network has no loops, or
- a feedback(recurrent) architecture, in which loops occur in the network because of feedback connections.



Neural Networks Generate Mappings

- Multilayered networks that associate vectors from one space to vectors of another space are called **heteroassociators**.
- Map or associate two different patterns with one another—one as input and the other as output. Mathematically we write,
$$f : R^n \rightarrow R^p.$$
- When neurons in a single field connect back onto themselves the resulting network is called an **autoassociator** since it associates a single pattern in R^n with itself.

